# MySQL 中的 MVCC 是什么？ - 面试鸭 | 2026最新面试题+详细答案解析

> ## Excerpt
>
> 2026最新后端、MySQL、数据库面试题，包含详细答案解析和练习题，25649+程序员已学习，已帮助10万+程序员通过校招/社招/实习面试，拿到offer！

## 回答重点

MVCC 全称 Multi-Version Concurrency Control，多版本并发控制。核心思想是**让读写操作互不阻塞。**

写操作修改数据时，MySQL 不会立即覆盖原有数据，而是生成新版本的记录。每个记录都保留了对应的版本号或时间戳。多版本之间串联起来就形成了一条版本链。

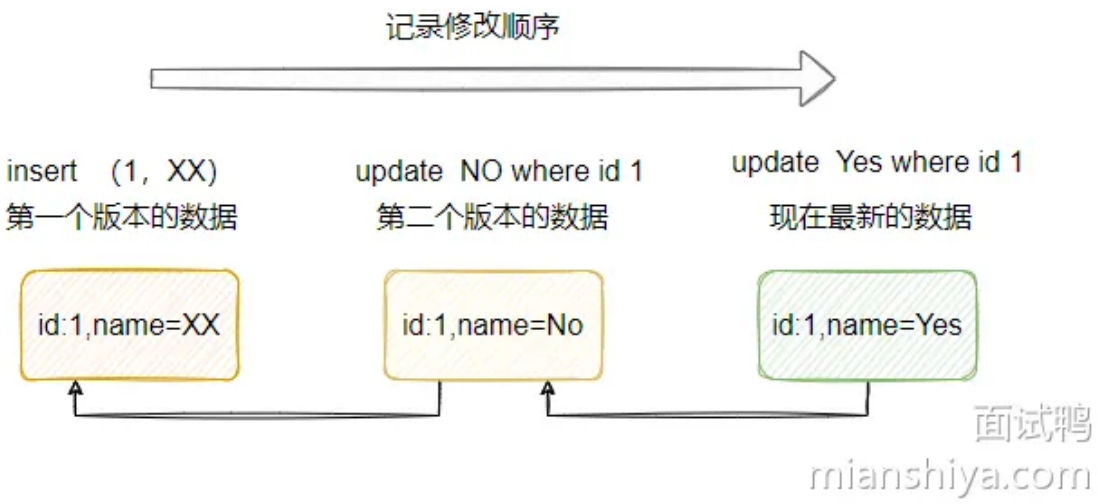
读操作（普通读）就可以无锁地根据事务启动时间去版本链上找到属于自己的那个版本，此时读(普通读)写操作不会阻塞。

具体实现是：InnoDB 里每条记录都有两个隐藏字段

- trx_id 记录最后修改这条数据的事务 ID，也就是当前事务 ID
- roll_pointer 指向 undo log 的指针。

每次 UPDATE 不会覆盖原数据，而是把旧值写到 undo log 里，新值写到数据页，roll_pointer 指向旧版本，这样就串成了一条**版本链。**

普通 SELECT 走的是快照读，不加锁，直接顺着版本链找到对自己可见的那个版本返回。写操作该怎么写怎么写，读写各走各的，并发性能直接拉满。

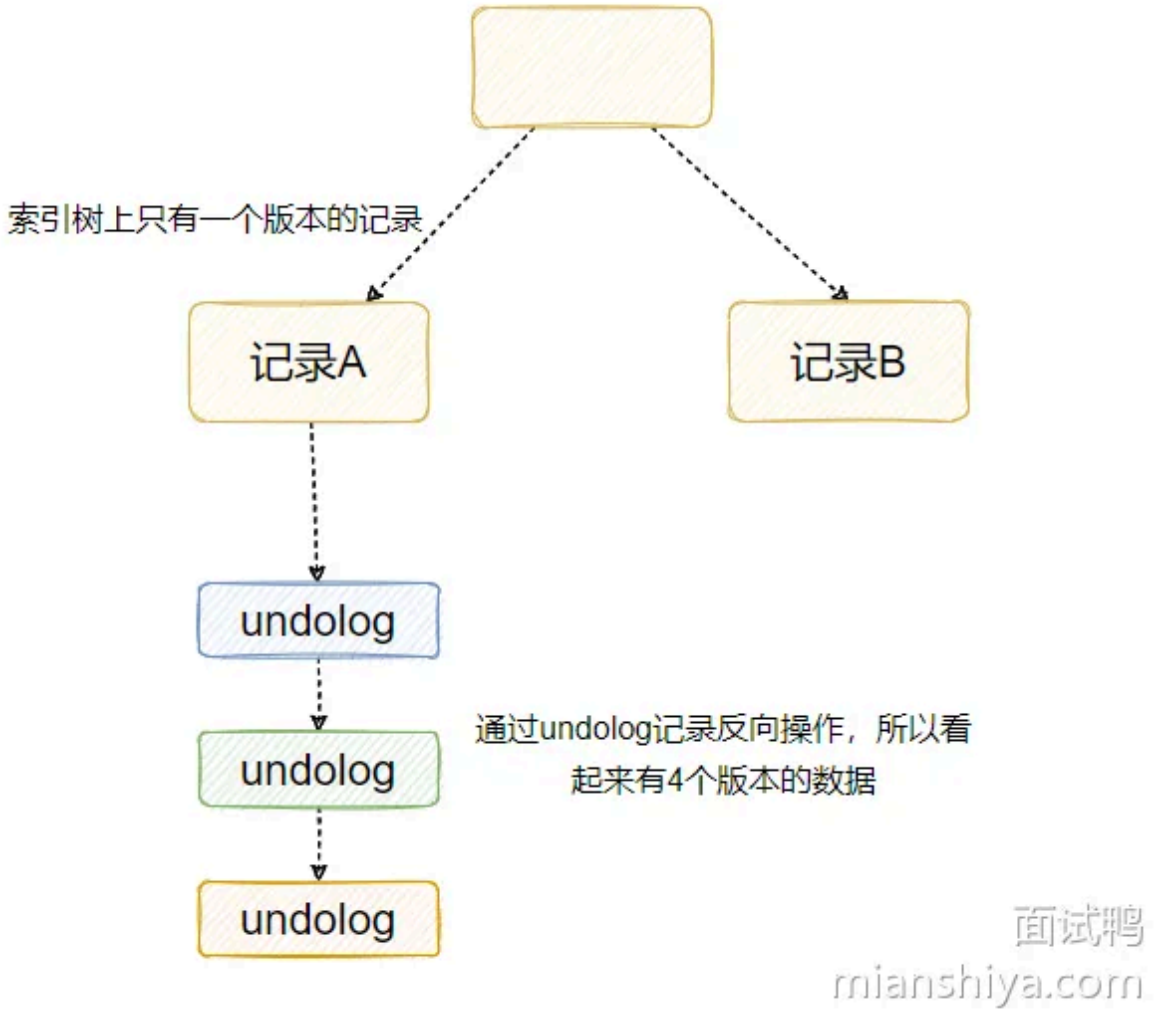



（为保持简短，简化了SQL语句，下面的内容也同样简化）

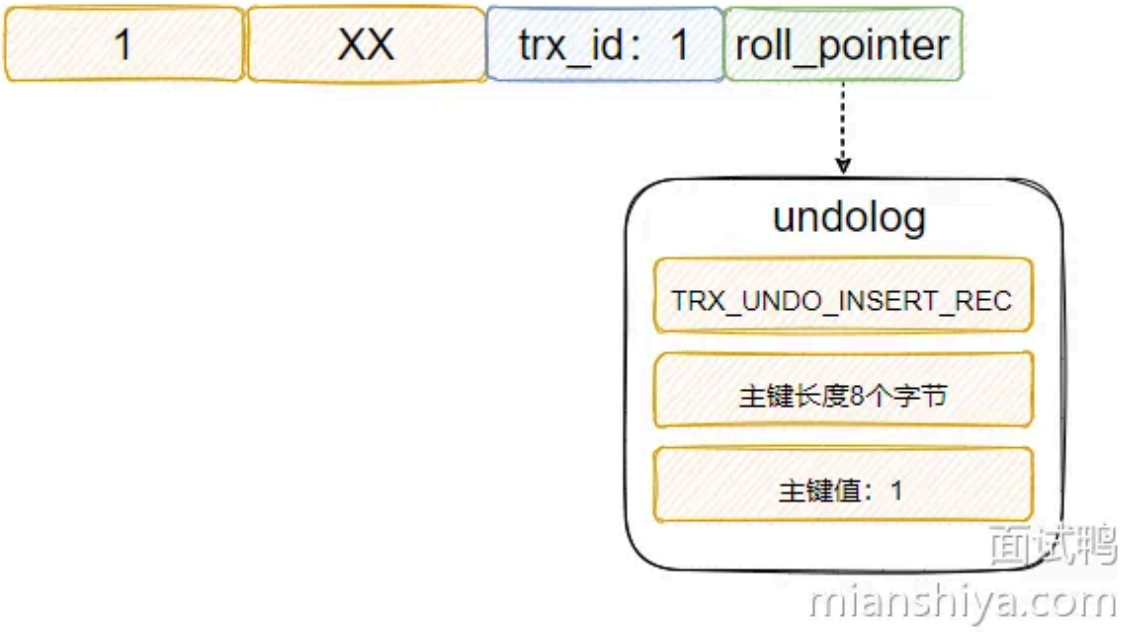
# 扩展知识

## Undo Log 和版本链

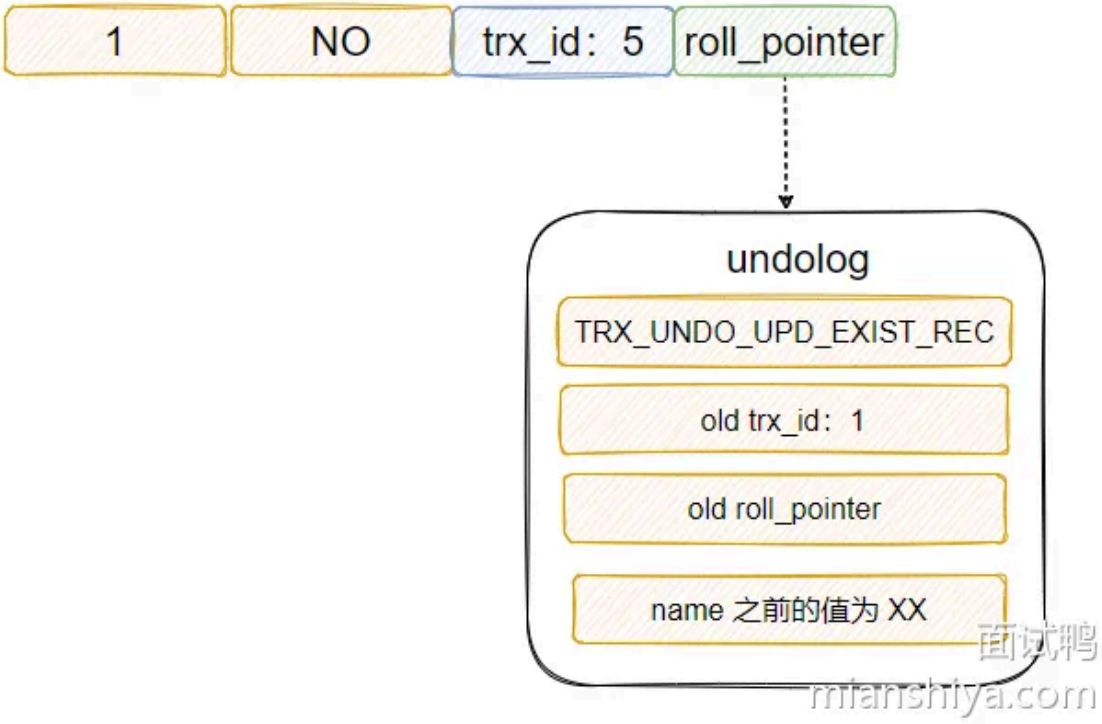
MVCC 的"多版本"并不是真的存了好几份数据。InnoDB 只在数据页上存最新版本，历史版本都在 undo log 里。undo log 里记的是反向操作，比如 UPDATE 就记"改之前是啥"，DELETE 就记"删之前是啥"。需要历史版本的时候，顺着 roll_pointer 往回推就行。




拿 `INSERT (1, XX)` 举例，插入成功后数据页上除了 ID=1、name=XX，还有 trx_id=1 和 roll_pointer。这条 undo log 类型是 `TRX_UNDO_INSERT_REC`，里面存了主键信息。回滚的时候根据主键找到这条记录删掉就行。

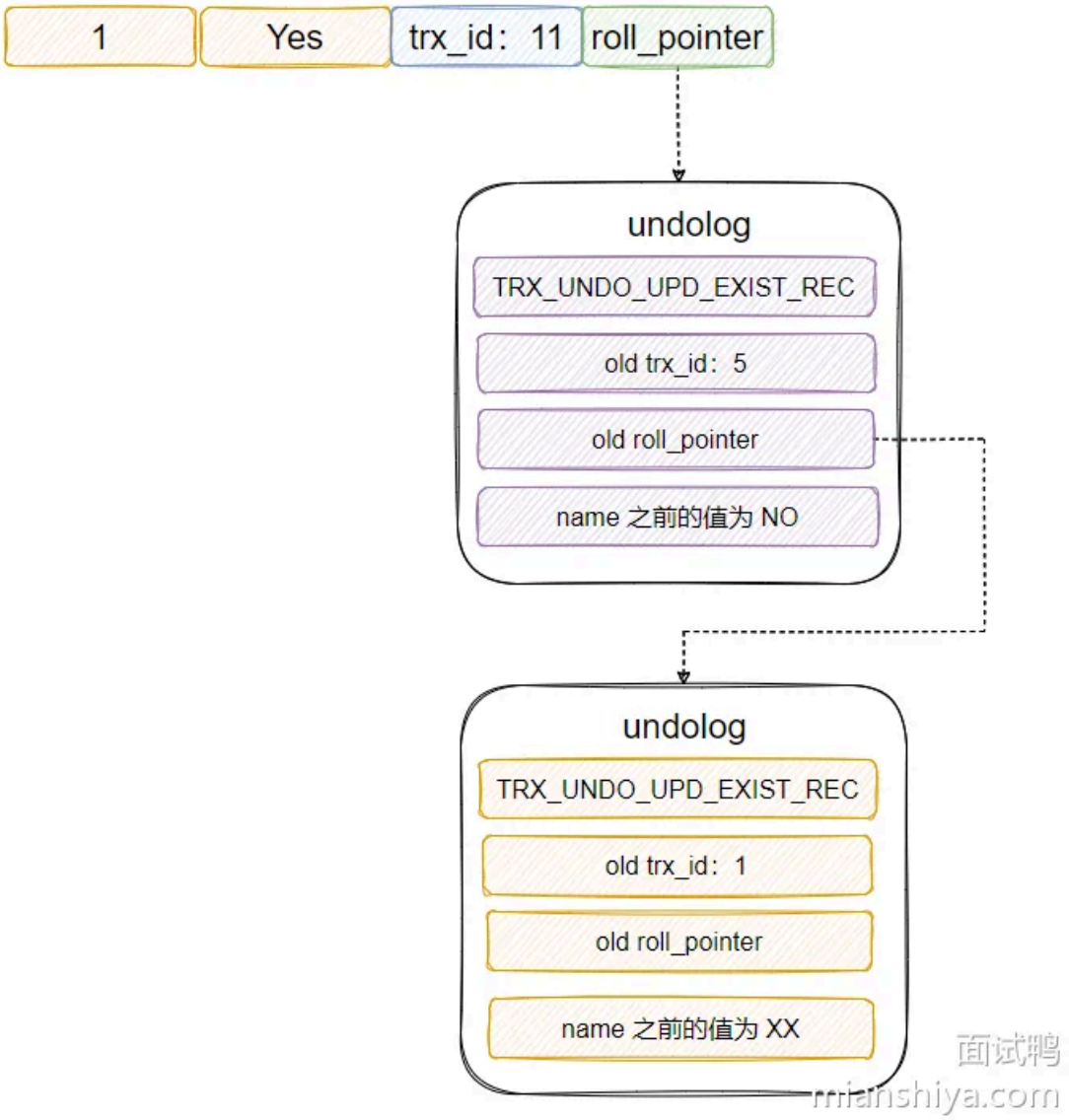



事务 1 提交后，另一个事务 5 执行 `UPDATE name='NO' WHERE id=1`：




INSERT 产生的 undo log 在事务提交后就回收了，因为不可能有别的事务会访问比插入更早的版本。UPDATE 产生的 undo log 类型是 `TRX_UNDO_UPD_EXIST_REC`，不会马上删，因为可能有别的事务需要读历史版本。

事务 5 提交后，事务 11 再执行 `UPDATE name='Yes' WHERE id=1`：




这样版本链就串起来了，id=1 这条记录加上两条 undo log，一共三个版本。

## ReadView 可见性判断

版本链有了，怎么判断哪个版本对当前事务可见？靠的是 **ReadView**。ReadView 里有四个关键字段：

1）creator_trx_id：当前事务 ID，只读事务这个值是 0

2）m_ids：生成 ReadView 时所有活跃事务的 ID 列表，就是已启动但还没提交的事务

3）min_trx_id：m_ids 里的最小值，也就是当前活跃ID之中的最小值

4）max_trx_id：下一个将被分配的事务 ID（事务 ID 是递增分配的，越后面申请的事务ID越大）

**对于可见版本的判断是从最新版本开始沿着版本链逐渐寻找老的版本，如果遇到符合条件的版本就返回。**

判断版本可见性的规则：

1）当前数据版本的 trx_id == creator_trx_id：说明修改这条数据的事务就是当前事务，所以可见

2）当前数据版本的 trx_id < min_trx_id：说明修改这条数据的事务在当前事务生成 readView 的时候已提交，所以可见。

3）当前数据版本的 trx_id >= max_trx_id：改这条数据的事务在生成 ReadView 之后才启动，不可见(结合事务ID递增来看)

4）min_trx_id <= 当前数据版本的 trx_id < max_trx_id：看 trx_id 在不在 m_ids 里，在就说明还没提交不可见，不在就说明已提交可见

## 读已提交和可重复读的区别

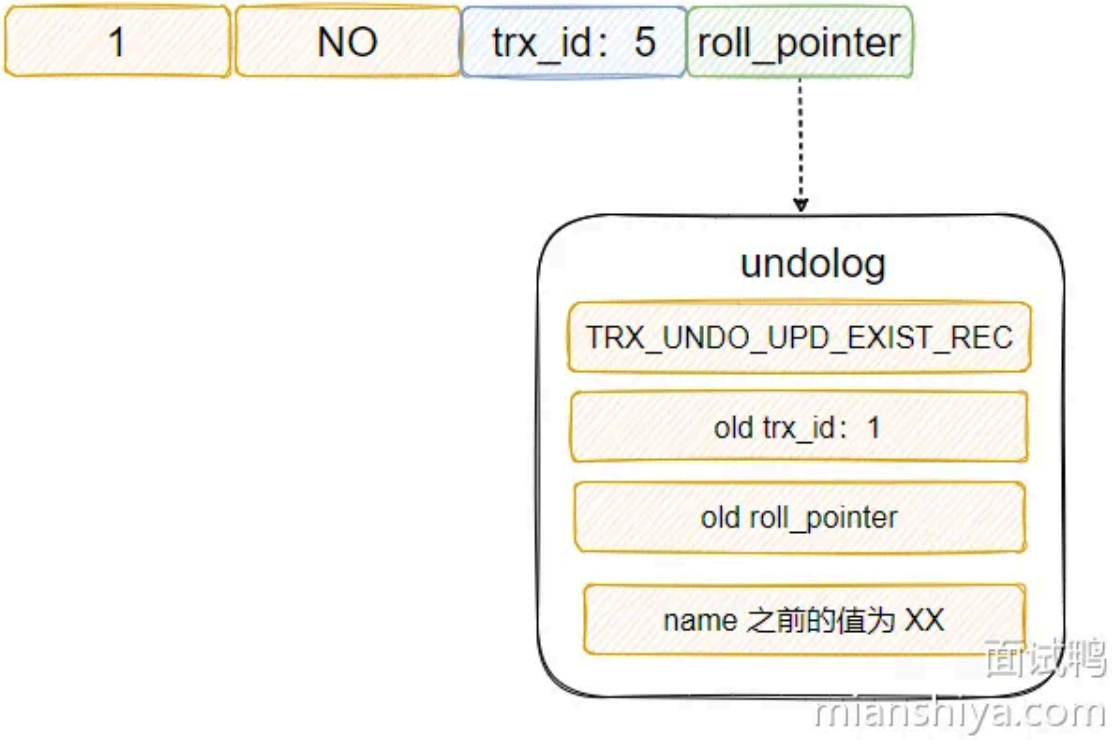
两个隔离级别判断版本可见性的逻辑一模一样，差别就一个：ReadView 生成时机不同。

**读已提交**：每次 SELECT 都重新生成 ReadView。别的事务提交了，下次查询就能看到。

**可重复读**：第一次 SELECT 生成 ReadView，后面的 SELECT 复用这个 ReadView。别的事务提交了也看不到，因为 ReadView 没变。

举个例子说明**读已提交的情况**。

假设事务 1 已提交，事务 5 正在执行还没提交，事务 6 也在执行没提交。此时一个新事务执行 `SELECT name WHERE id=1`，由于查询的是 ID 为 1 的记录，所以先找到 ID 为 1 的这条记录，此时的版本如下：




此时 ReadView：

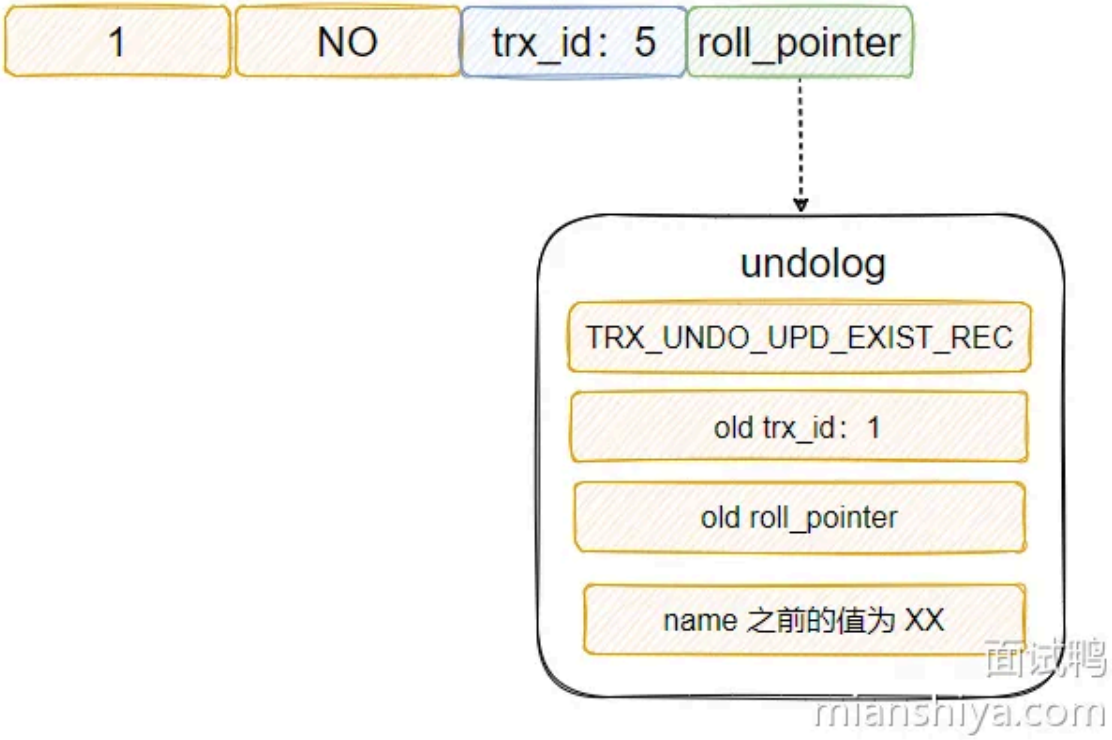
- creator_trx_id=0，因为一个事务只有当有修改操作的时候才会被分配事务 ID。
- m_ids=[5,6]，这两个事务都未提交，为活跃的。
- min_trx_id=5
- max_trx_id=7，因为最新分配的事务 ID 为 6，那么下一个就是7，事务 ID 是递增分配的

最新版本 trx_id=5，在 m_ids 里，不可见。顺着 roll_pointer 通过版本链找到上一个版本 trx_id=1 的版本，比 min_trx_id 小，说明在生成 readView 的时候已经提交，可见。因此返回 name=XX。

**然后事务 5 提交**，再次查询生成新的 ReadView：

- creator_trx_id 为 0，因为还是没有修改操作。
- m_ids 为 [6]，因为事务5提交了。
- min_trx_id=6
- max_trx_id=7，此时没有新的事务申请。

同样还是查询的是 ID 为 1 的记录，所以还是先找到 ID 为 1 的这条记录，此时的版本如下(和上面一样，没变)：




此时最新版本 trx_id=5，比 min_trx_id=6 小，说明事务已经提交了，可见。返回 name=NO。

**这就是读已提交的 MVCC 操作**，同一个事务两次查询结果不一样，这就是不可重复读。

**如果现在的隔离级别是可重复读。**

那么第二次查询 **复用** 第一次的 ReadView，m_ids 还是 [5,6]，min_trx_id 还是 5，所以 trx_id=5 的版本还是不可见，两次都返回 name=XX。

所以可重复读和读已提交的 MVCC 判断版本的过程是一模一样的，**唯一的差别在生成 readView 上。**

**读已提交**每次查询都会重新生成一个新的 readView ，而**可重复读**在第一次生成 readView 之后的所有查询都共用同一个 readView，所以一个事务里面不论有几次 select ，其实看到的都是同一个 readView，所以叫**可重复读**。

## 快照读和当前读

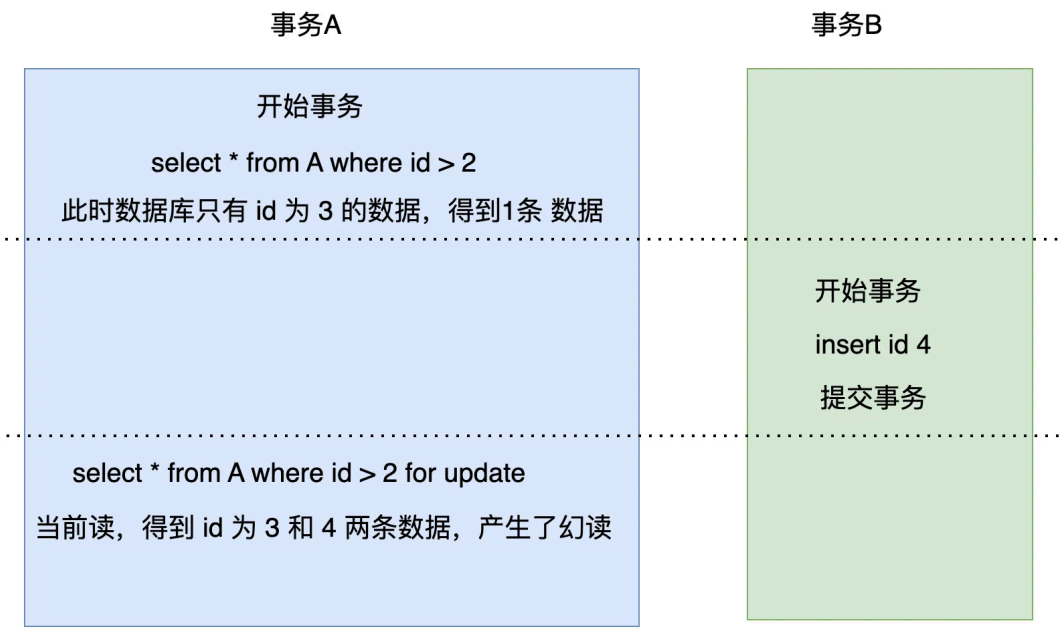
**快照读**就是普通 SELECT，走 MVCC，读的是历史快照，不加锁。

**当前读**是 `SELECT ... FOR UPDATE`、`SELECT ... LOCK IN SHARE MODE`、UPDATE、DELETE 这些，必须读最新版本并且加锁。道理很简单，你要 UPDATE 一条数据，不看最新版本怎么行？万一别的事务刚删了，你还 UPDATE 成功了那不乱套了。

当前读会加 Next-Key Lock，锁住记录和间隙，防止别的事务插入新数据。

## 可重复读能完全避免幻读吗

不能。快照读和当前读混用的时候会出问题：




面试鸭
mianshiya.com

事务 1 先快照读，拿到的是事务启动时的快照。事务 2 插入一条新数据提交了。事务 1 再当前读，因为当前读要获取最新版本，所以能看到事务 2 插入的数据。两次查询结果行数不一样，幻读就出现了。

解决办法是从头就用 `SELECT ... FOR UPDATE`，加上锁别的事务就插不进来了。

# 面试官追问

## 提问：undo log 什么时候会被清理？

回答：INSERT 产生的 undo log 在事务提交后立刻就能清理，因为没人会去读插入之前的版本。UPDATE 和 DELETE 产生的 undo log 要等到没有任何事务需要这个版本才能清。InnoDB 后台有个 purge 线程专门干这事，它会根据当前活跃事务的 ReadView 判断哪些 undo log 可以安全删除。长事务会导致 undo log 堆积，这也是为什么要尽量避免长事务。

## 提问：MVCC 能不能解决写写冲突？

回答：不能。MVCC 解决的是读写冲突，让普通读不阻塞写、写不阻塞普通读。写写冲突还是要靠锁来解决。两个事务同时 UPDATE 同一行，后执行的那个要等先执行的提交或回滚才能拿到锁。

## 提问：为什么 ReadView 里要记 max_trx_id？只用 m_ids 不行吗？

回答：不行。m_ids 只记录了生成 ReadView 那一刻的活跃事务，但事务 ID 是递增的，之后新启动的事务 ID 肯定比 m_ids 里的都大。如果只用 m_ids 判断，新启动事务修改的数据就可能被当成已提交的看到。max_trx_id 就是用来挡住这些后来者的，trx_id >= max_trx_id 一律不可见。

## 提问：只读事务的 creator_trx_id 是 0，那它自己修改的数据怎么判断可见？

回答：只读事务不会有修改操作，所以不存在"自己修改的数据"这个场景。一旦有写操作，InnoDB 会给这个事务分配一个真正的事务 ID，creator_trx_id 就不是 0 了。